**FACULDADE SENAI FATESG**
**ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Caio Abreu
Cassiano Abreu
Enzo Aquino
Gabriel Naoki
Wyllian Mariano

# **Event-Driven Architecture (EDA):** Descrição técnica de um sistema de pedidos em Java

Disciplina: Desenvolvimento de Sistemas Distribuídos - 4º período
Profº Ramyres Aquino

Goiânia-Go
2026/2

## 1 - Introdução

Este documento explica o código do projeto desenvolvido pelo grupo para o seminário de Desenvolvimento de Sistemas Distribuídos, cujo tema é Arquitetura Orientada a Eventos (Event-Driven Architecture – EDA). O projeto é um sistema de pedidos em Java puro, sem bibliotecas externas (11 arquivos, 475 linhas), que mostra em uma demonstração no terminal como serviços independentes se comunicam apenas por eventos.

**Repositório:**
https://github.com/GabrielNaokiUT/EventDrivenArchitecture_DesenvolvimentoSistemasDistribuidos

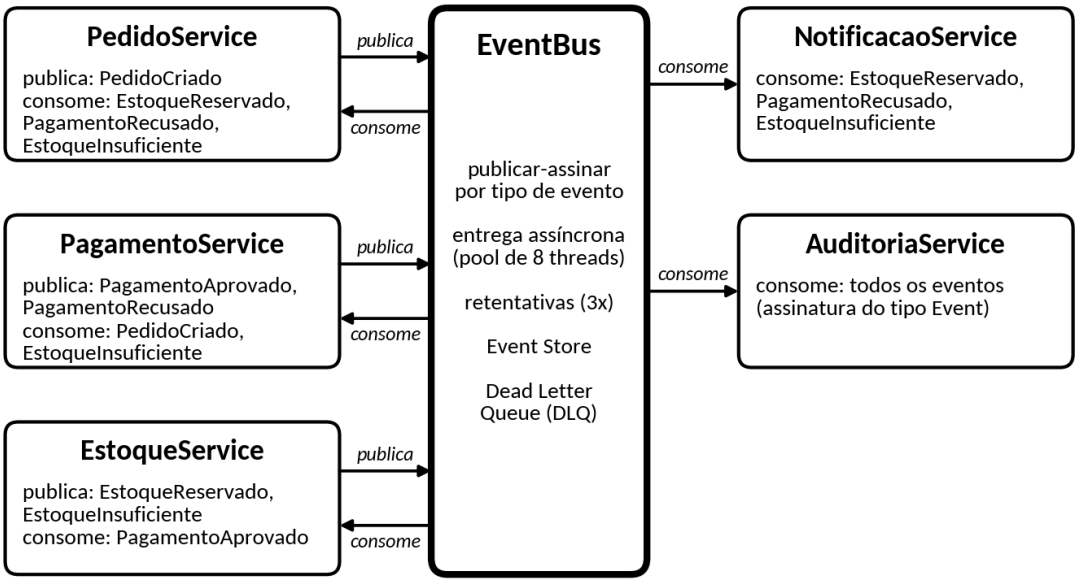
## 2 - O que é EDA?

Em uma arquitetura orientada a eventos, os componentes não se chamam diretamente: eles trocam **eventos** por meio de um intermediário.

- **Evento:** um fato que já aconteceu, como PedidoCriado ou PagamentoAprovado.
- **Producer (produtor):** quem publica o evento. Ele não sabe quem vai reagir.
- **Consumer (consumidor):** quem se inscreve em um tipo de evento e reage quando ele chega.
- **Broker (event bus):** o intermediário que recebe os eventos e os entrega aos consumidores. No projeto, é a classe EventBus.
- **Desacoplamento:** produtor e consumidor não se conhecem. Dá para adicionar ou remover um consumidor (como a auditoria) sem alterar os outros serviços, e a falha de um não derruba os demais.

**Observação:** o EventBus roda em memória, na mesma JVM, e simula o broker para fins didáticos. Em produção ele seria substituído por Kafka ou RabbitMQ, com os serviços em processos (ou máquinas) diferentes.

## 3 - Arquitetura e fluxo de eventos




Na Figura 1, a seta para o EventBus indica publicação e a seta para o serviço indica consumo. **Fluxo de um pedido:** PedidoCriado → PagamentoAprovado → EstoqueReservado → pedido confirmado e cliente notificado. Se o pagamento for recusado ou faltar estoque, o pedido é cancelado e, no caso do estoque, o pagamento é estornado (transação de compensação). Nenhum serviço conhece outro: todos conhecem apenas o EventBus e os tipos de evento.

## 4 - Casos de uso no mercado

A EDA é usada quando várias partes do sistema precisam reagir ao mesmo acontecimento sem depender umas das outras:

- **E-commerce:** um pedido criado dispara pagamento, reserva de estoque, nota fiscal e e-mail ao cliente, cada um em seu serviço (é o caso deste projeto).
- **Bancos e fintechs:** cada transação gera eventos usados por antifraude, extrato, notificações e auditoria.
- **Logística:** atualizações de rastreamento de entregas e de frota alimentam painéis e alertas em tempo real.
- **IoT e monitoramento:** leituras de sensores são publicadas como eventos e tratadas por vários consumidores.

## 5 - Estrutura e componentes

O código está em src/main/java/eda/ e usa Java 21 e Maven. O Quadro 1 resume as classes.

| Classe | Função |
|---|---|
| core/Event, Envelope | Contrato do evento e envelope com id e data/hora da publicação. |
| core/EventBus | Broker: assinatura por tipo de evento, entrega assíncrona (pool de 8 threads), até 3 tentativas, Event Store e DLQ. |
| core/Log | Log com hora, thread e serviço. |
| eventos/Eventos | Records PedidoCriado, PagamentoAprovado, PagamentoRecusado, EstoqueReservado e EstoqueInsuficiente (levam cliente, produto, quantidade e valor). |
| servicos/PedidoService | Producer de PedidoCriado; atualiza o status do pedido (CRIADO, CONFIRMADO ou CANCELADO). |
| servicos/PagamentoService | Aprova pedidos de até R$ 5.000,00, recusa os maiores e estorna o pagamento se faltar estoque (compensação). |
| servicos/EstoqueService | Reserva o estoque (Notebook 5, Mouse 50, Monitor 1); é synchronized e idempotente (ignora evento repetido). |
| servicos/NotificacaoService | Simula o e-mail ao cliente e permite forçar falhas para mostrar retry e DLQ. |
| servicos/AuditoriaService | Assina todos os eventos e registra cada um. |
| App | Menu do terminal com os 8 cenários da demonstração. |

## 6 - Demonstração (live preview)

Requisitos: JDK 21 ou superior e Maven. Na pasta do projeto:

```
mvn clean package
java -jar target/eda-seminario.jar
mvn compile exec:java
```

O primeiro comando compila e gera o .jar, o segundo executa o programa e o terceiro é uma alternativa que executa direto pelo Maven, sem usar o .jar. O programa abre um menu: digite o número do cenário e pressione Enter (0 encerra). O estoque não é reposto entre os cenários, e o cenário 8 deve ser executado depois de outros, pois a lista começa vazia.

| Nº | Cenário | O que mostra |
|---|---|---|
| **1** | Pedido com sucesso | Fluxo completo: pagamento aprovado, estoque reservado, pedido CONFIRMADO e e-mail enviado. |
| **2** | Pagamento recusado | Pedido acima de R$ 5.000,00 é recusado e fica CANCELADO. |
| **3** | Sem estoque | Pagamento aprovado, mas o estoque não basta: o pagamento é estornado (compensação) e o pedido é cancelado. |
| **4** | 3 pedidos ao mesmo tempo | O publish() retorna na hora (status CRIADO) e o processamento é assíncrono; depois todos convergem (consistência eventual). |
| **5** | Falha temporária na notificação | O envio falha 2 vezes e funciona na 3ª tentativa (retry). |
| **6** | Falha permanente na notificação | Depois de 3 falhas o evento vai para a Dead Letter Queue; o pedido continua CONFIRMADO. |
| **7** | Evento duplicado | O mesmo PagamentoAprovado é reenviado e o estoque ignora a repetição (idempotência). |
| **8** | Ver Event Store e DLQ | Lista todos os eventos publicados e as mensagens da DLQ. |

## 7 - Limitações e conclusão

- O EventBus é em memória e roda em uma única JVM: os serviços não são processos separados e não se comunicam pela rede.
- Event Store, DLQ, pedidos e estoque se perdem ao encerrar o programa, e a DLQ não é reprocessada.
- O pool de threads não garante a ordem entre eventos, e só o EstoqueService tem controle explícito de idempotência.

Mesmo assim, o projeto cobre os principais elementos de EDA com código simples e executável: eventos, producer, consumer, broker, desacoplamento, assincronia, consistência eventual, compensação, retry, DLQ, Event Store e idempotência. Para um ambiente realmente distribuído, bastaria trocar o EventBus por um broker como Kafka ou RabbitMQ.

## 8 - Participação do grupo

Cada parte foi enviada ao repositório em uma branch própria, por pull request.

| Integrante | GitHub | Parte | Arquivos |
|---|---|---|---|
| Caio Abreu | Caio4breu | Núcleo do barramento | Event, Envelope, Log, EventBus |
| Cassiano Abreu | Nomscodes | Pagamento e estoque | PagamentoService, EstoqueService |
| Enzo Aquino | EnzoAkino | Eventos, producer e auditoria | Eventos, PedidoService, AuditoriaService |
| Gabriel Naoki | GabrielNaokiUT | Repositório e documentação | Estrutura de pastas, documentação, pom.xml, .gitignore, README.md |
| Wyllian Mariano | wyllianmn | Notificação e demo | NotificacaoService, App |